

PROJECT REPORT

Post-Quantum Cryptography Based TLS Communication

Implementation and Performance Evaluation using OpenSSL, liboqs and the OQS Provider

A Practical Study of Quantum-Safe TLS Handshake
TLS Communication using PQC Algorithms with Wireshark Analysis

Submitted by	Indrajit Singh
Environment	Ubuntu Linux on VMware
Tools Used	OpenSSL 3.x, liboqs, OQS Provider, Wireshark
Topic	Post-Quantum Cryptography in TLS
Year	2026

Table of Contents

1. Introduction	3
2. Why the Quantum Threat Exists	3
3. Need for Post-Quantum Cryptography	3
4. Selected PQC Algorithms	3
5. Environment Setup	4
6. TLS Server and Client Setup	4
7. Wireshark Packet Analysis	5
8. Important Screenshots and Explanation	5
8.1 OQS Provider Setup	5
8.2 liboqs Build Process	5
8.3 OQS Provider Compilation	5
8.4 Available ML-DSA / SLH-DSA Signature Algorithms	6
8.5 Available ML-KEM Key Encapsulation Algorithms	6
8.6 Server Private Key Generation using ML-KEM-768	7
8.7 Self-Signed PQC Certificate Creation	7
8.8 TLS Server Startup	8
8.9 TLS Session Established with Session Ticket	8
8.10 Client-Server Encrypted Communication	9
8.11 Initial TLS Packet Capture	9
8.12 Cipher Suite Negotiation Verification	10
8.13 Full TLS 1.3 Handshake Analysis	10
9. Performance Evaluation and Comparison	11
10. Conclusion	12
11. Future Scope	12

1. Introduction

Transport Layer Security (TLS) is widely used to protect communication over networks and the internet. It secures sensitive information such as login credentials, banking data, and confidential communication between clients and servers.

Traditional TLS mainly depends on cryptographic algorithms like RSA and ECC. Although these algorithms are secure against classical computers, the development of quantum computing creates a major future threat. Quantum computers may be capable of breaking traditional cryptographic algorithms much faster using advanced algorithms such as Shor's algorithm.

To overcome this issue, researchers introduced Post-Quantum Cryptography (PQC), which focuses on cryptographic algorithms that can resist attacks from both classical and quantum computers. In this project, PQC-based TLS communication was implemented using OpenSSL, liboqs, and the OQS provider.

2. Why the Quantum Threat Exists

Current encryption systems are based on mathematical problems that are difficult for classical computers to solve. RSA depends on prime factorization, while ECC depends on elliptic curve mathematics.

Quantum computers use a completely different computational model. If large-scale quantum computers become available in the future, they may solve these mathematical problems in a very short time. This creates a serious risk because encrypted communication that is secure today may become vulnerable in the future.

Due to this reason, cybersecurity researchers are developing Post-Quantum Cryptography algorithms to provide future-ready protection.

3. Need for Post-Quantum Cryptography

The main objective of Post-Quantum Cryptography is to secure communication systems against future quantum attacks. Organizations handling sensitive data need long-term protection because attackers may store encrypted traffic today and decrypt it later when quantum computers become practical.

PQC algorithms are designed in such a way that they remain secure even against quantum-based attacks. Integrating PQC with TLS communication helps improve future security while maintaining secure encrypted communication between systems.

4. Selected PQC Algorithms

This implementation used PQC support provided by liboqs and the OQS provider. Hybrid cryptography was used where classical TLS algorithms work together with post-quantum algorithms.

Popular PQC algorithms such as Kyber (used for key encapsulation) and Dilithium (used for digital signatures) are commonly used in modern PQC research because they provide strong security and practical performance. Hybrid TLS communication helps maintain compatibility with existing systems while improving resistance against future quantum attacks.

5. Environment Setup

The project was implemented inside Ubuntu Linux running on VMware. Several tools and libraries were installed for successful implementation. The setup process was carried out step by step to ensure that all components worked together correctly.

Technologies and Tools Used

Technology / Tool	Purpose
Ubuntu Linux	Host operating system for running the entire setup
OpenSSL 3.x	Provides core TLS implementation and cryptographic library
liboqs	Open Quantum Safe library that implements PQC algorithms
OQS Provider	Plugs liboqs into OpenSSL 3.x as a provider
Ninja Build System	Fast build tool used during compilation
CMake	Build configuration system used by liboqs and OQS Provider
Wireshark	Network protocol analyzer used to capture TLS traffic

The repositories were cloned successfully from GitHub and compiled using CMake and Ninja. After compilation, the OQS Provider was registered with OpenSSL so that PQC algorithms could be used in TLS communication.

6. TLS Server and Client Setup

After successful installation of liboqs and the OQS provider, TLS communication was configured using OpenSSL commands. The configuration was done in such a way that the server and client both used PQCEnabled cryptographic primitives during the TLS handshake.

A private key, certificate signing request (CSR), and self-signed certificate were generated successfully. The TLS server was started locally, and the client established encrypted communication with the server over the loopback interface (127.0.0.1).

This step confirmed that secure communication between the server and client was working properly, and that the OQS Provider had been correctly integrated into the TLS pipeline

7. Wireshark Packet Analysis

Wireshark was used to monitor and analyze TLS traffic generated between the client and server. Packets were captured on the loopback interface and filtered using the `tls` display filter to isolate TLS records.

The packet capture showed important TLS communication stages such as:

- Client Hello — client advertises supported cipher suites and key shares.
- Server Hello — server selects parameters and replies with its own key share.
- Change Cipher Spec — both peers switch to the negotiated symmetric keys.
- Application Data — the actual encrypted payload exchanged after handshake.

These packets confirm successful TLS handshakes and encrypted communication between both systems. The Wireshark analysis also demonstrated how TLS encrypts application data during transmission, so that even on the loopback interface the actual content remains unreadable in the packet bytes.

8. Important Screenshots and Explanation

This section presents the most important screenshots captured during the implementation. Each screenshot is followed by a brief explanation that describes what is being shown and why it is significant for the project.

8.1 OQS Provider Setup

The first step of the implementation was to clone the OQS Provider source code from the official Open Quantum Safe GitHub repository. Git was used to fetch all the objects and write the repository contents into a local directory named `oqs-provider`. After cloning, the working directory was changed into this folder to prepare for the build. This step laid the foundation for compiling the provider that bridges `liboqs` with OpenSSL.

8.2 liboqs Build Process

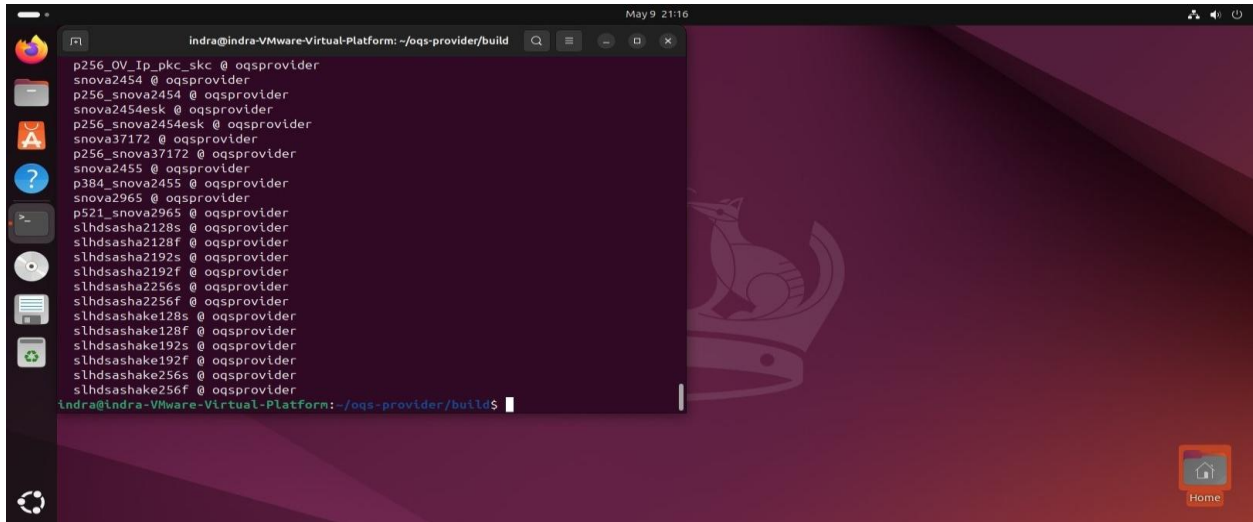
After cloning, the next step was to build the `liboqs` library, which is the actual implementation of PostQuantum cryptographic algorithms. The CMake configuration completed successfully and the Ninja build system linked the final executable (`tests/dump_alg_info`). The build output confirmed that OpenSSL 3.0.13 was detected, threads were enabled, and all 2656 compilation targets were processed without errors.

8.3 OQS Provider Compilation

Once `liboqs` was built, the OQS Provider itself had to be configured against it. The CMake configuration step for the OQS Provider successfully located the system OpenSSL installation at `/usr/lib/x86_64-linux-gnu/libcrypto.so` and the previously built `liboqs` library. The messages “Configuring done” and “Generating done” confirmed that the OQS Provider was ready to be compiled and integrated with OpenSSL as a provider plugin.

8.4 Available ML-DSA / SLH-DSA Signature Algorithms

Once the OQS Provider was loaded into OpenSSL, the next step was to verify that the post-quantum digital signature algorithms were registered correctly. Running the algorithm listing command produced a long list of providers ending with @ oqsprovider, including hybrid SNOVA variants such as snova2454, snova37172, snova2455, snova2965 and their classical-hybrid forms (for example p256_snova2454 and p384_snova2455). The stateless hash-based SLH-DSA family is also visible, with parameter sets slhdsasha2-128s/f, slhdsasha2-192s/f, slhdsasha2-256s/f and the SHAKE variants slhdsashake128s/f, slhdsashake192s/f, slhdsashake256s/f. Their presence in the listing confirms that the OQS Provider successfully exposes the NIST PQC signature algorithms to OpenSSL.

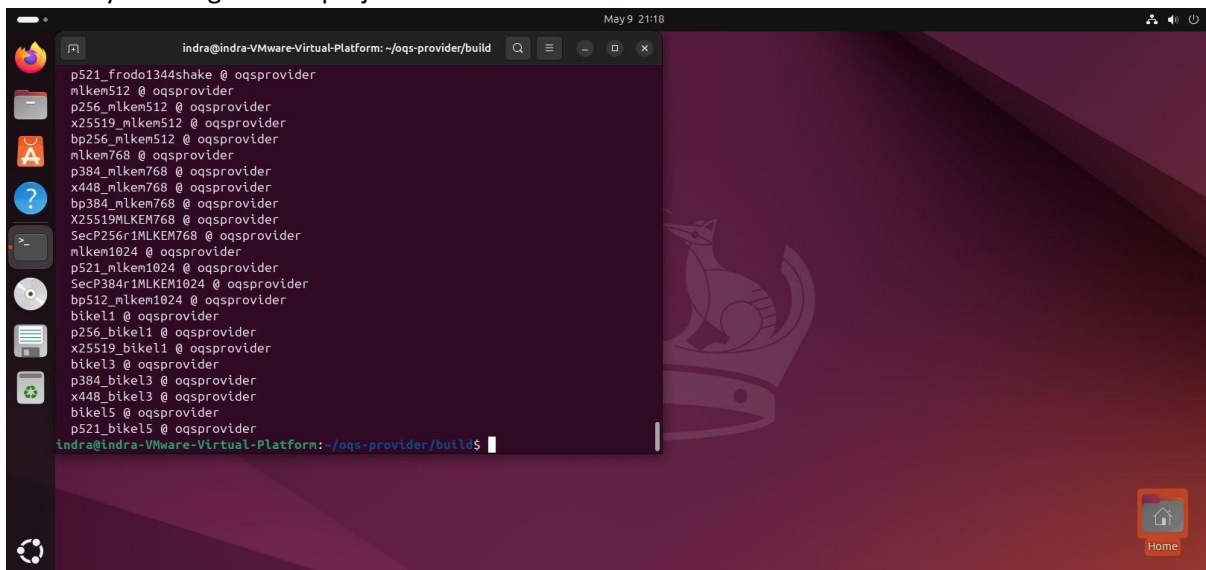


```
indra@indra-VMware-Virtual-Platform: ~/oqs-provider/build
p256_OV_ip_pkc_skc @ oqsprovider
snova2454 @ oqsprovider
p256_snova2454 @ oqsprovider
snova2454esk @ oqsprovider
p256_snova2454esk @ oqsprovider
snova37172 @ oqsprovider
p256_snova37172 @ oqsprovider
snova2455 @ oqsprovider
p384_snova2455 @ oqsprovider
snova2965 @ oqsprovider
p521_snova2965 @ oqsprovider
slhdsasha2128s @ oqsprovider
slhdsasha2128f @ oqsprovider
slhdsasha2192s @ oqsprovider
slhdsasha2192f @ oqsprovider
slhdsasha2256s @ oqsprovider
slhdsasha2256f @ oqsprovider
slhdsashake128s @ oqsprovider
slhdsashake128f @ oqsprovider
slhdsashake192s @ oqsprovider
slhdsashake192f @ oqsprovider
slhdsashake256s @ oqsprovider
slhdsashake256f @ oqsprovider
indra@indra-VMware-Virtual-Platform: ~/oqs-provider/build$
```

Figure: List of post-quantum digital signature algorithms (SNOVA and SLH-DSA family) registered through the OQS Provider.

8.5 Available ML-KEM Key Encapsulation Algorithms

In addition to digital signatures, the OQS Provider also exposes post-quantum Key Encapsulation Mechanisms (KEMs). The listing shows the full ML-KEM family standardised by NIST — mlkem512, mlkem768 and mlkem1024 — together with their hybrid variants that combine ML-KEM with a classical algorithm such as p256_mlkem512, x25519_mlkem512, p384_mlkem768, x448_mlkem768, bp384_mlkem768, X25519MLKEM768, SecP256r1MLKEM768, p521_mlkem1024, SecP384r1MLKEM1024 and bp512_mlkem1024. Additional BIKE variants (bikel1, bikel3, bikel5 and their hybrid forms) are also available. These hybrid KEMs are the building blocks used by TLS 1.3 for the actual quantum-safe key exchange in this project.

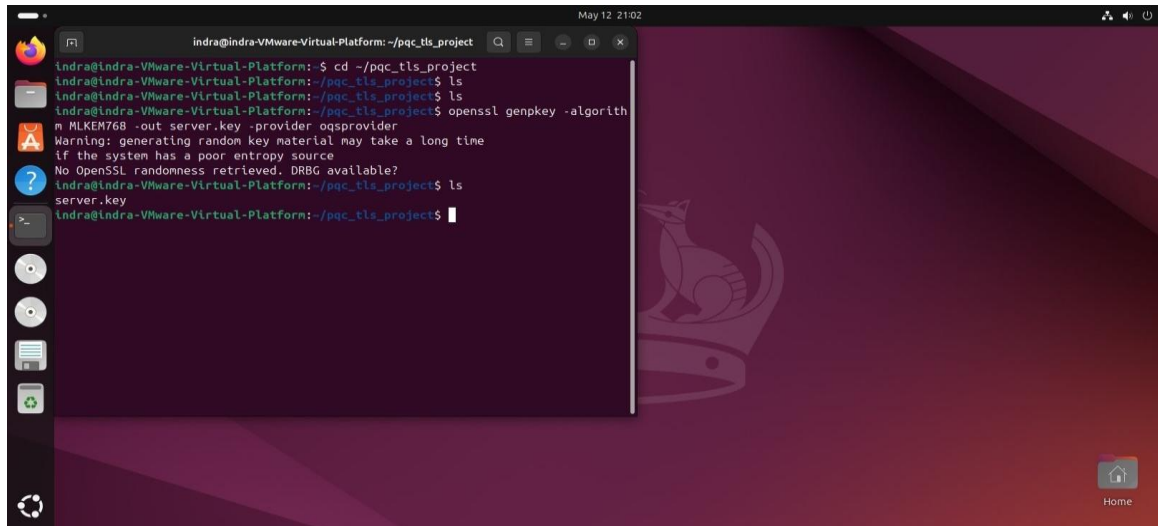


```
indra@indra-VMware-Virtual-Platform: ~/oqs-provider/build
p521_frodo1344shake @ oqsprovider
mlkem512 @ oqsprovider
p256_mlkem512 @ oqsprovider
x25519_mlkem512 @ oqsprovider
bp256_mlkem512 @ oqsprovider
mlkem768 @ oqsprovider
p384_mlkem768 @ oqsprovider
x448_mlkem768 @ oqsprovider
bp384_mlkem768 @ oqsprovider
X25519MLKEM768 @ oqsprovider
SecP256r1MLKEM768 @ oqsprovider
mlkem1024 @ oqsprovider
p521_mlkem1024 @ oqsprovider
SecP384r1MLKEM1024 @ oqsprovider
bp512_mlkem1024 @ oqsprovider
bikel1 @ oqsprovider
p256_bikel1 @ oqsprovider
x25519_bikel1 @ oqsprovider
bikel3 @ oqsprovider
p384_bikel3 @ oqsprovider
x448_bikel3 @ oqsprovider
bikel5 @ oqsprovider
p521_bikel5 @ oqsprovider
indra@indra-VMware-Virtual-Platform: ~/oqs-provider/build$
```

Figure: List of post-quantum and hybrid Key Encapsulation Mechanisms (ML-KEM and BIKE) registered through the OQS Provider.

8.6 Server Private Key Generation using ML-KEM-768

With the provider in place, a post-quantum private key was generated for the TLS server using the command `openssl genpkey -algorithm MLKEM768 -out server.key -provider oqsprovider`. The command produced a warning about random key material generation taking time on systems with poor entropy, which is expected on a virtual machine, and then completed successfully. The directory listing afterwards confirms that the file `server.key` was created in the project folder. This key is the private half of the MLKEM-768 key pair that the server will later use during the TLS handshake

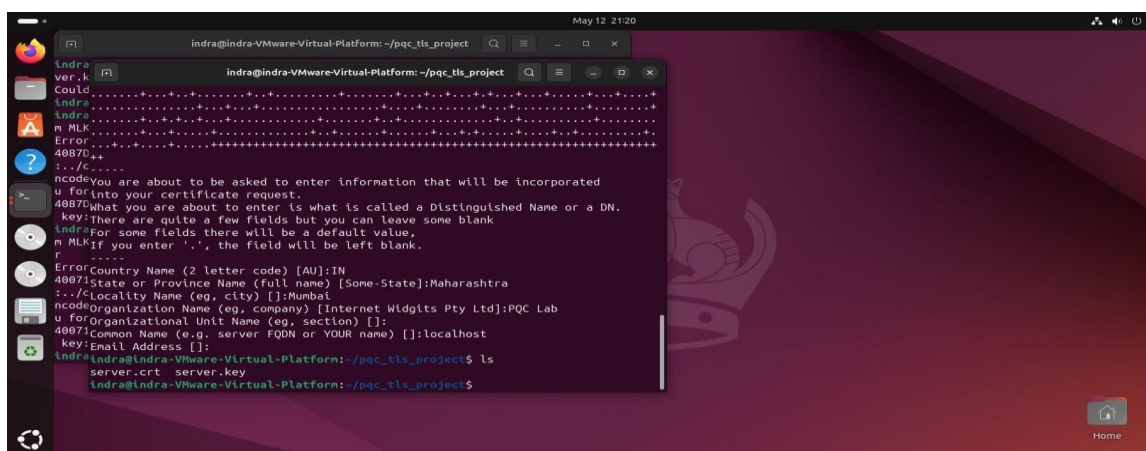


```
indra@indra-VMware-Virtual-Platform: ~/pqc_tls_project
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$ ls
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$ openssl genpkey -algorithm MLKEM768 -out server.key -provider oqsprovider
Warning: generating random key material may take a long time if the system has a poor entropy source
No OpenSSL randomness retrieved. DRBG available?
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$ ls
server.key
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$
```

Figure: Generation of an ML-KEM-768 post-quantum private key (`server.key`) using OpenSSL with the OQS Provider.

8.7 Self-Signed PQC Certificate Creation

After the private key was generated, a Certificate Signing Request (CSR) was created using `openssl req new`, where the Distinguished Name fields were filled in interactively. The values entered were Country = IN, State = Maharashtra, Locality = Mumbai, Organisation = PQC Lab and Common Name = localhost, which matches the loopback address that the local TLS server will listen on. The CSR was then signed with the server's own private key using `openssl x509 -req -signkey` to produce a self-signed certificate. The final directory listing shows both `server.key` and `server.crt` present, meaning the server now has the credentials needed to start a TLS service.



```
indra@indra-VMware-Virtual-Platform: ~/pqc_tls_project
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$ openssl req new
Could not read config file: /etc/openssl/openssl.cnf
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:IN
State or Province Name (full name) [Some-State]:Maharashtra
Locality Name (eg, city) []:Mumbai
Organization Name (eg, company) [Internet Widgits Pty Ltd]:PQC Lab
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:localhost
key:
Email Address []:
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$ ls
server.crt server.key
indra@indra-VMware-Virtual-Platform:~/pqc_tls_project$
```

Figure: Interactive CSR generation and self-signed certificate creation, producing `server.crt` alongside `server.key`

8.8 TLS Server Startup

With both the private key and the certificate ready, the TLS server was started using `openssl s_server -key server.key -cert server.crt -accept 4433`. OpenSSL printed “Using default temp DH parameters” and then “ACCEPT”, which indicates that the server has successfully bound to TCP port 4433 on the loopback interface and is now waiting for an incoming TLS client. At this point the PQC-enabled OpenSSL stack is fully running and listening for handshakes

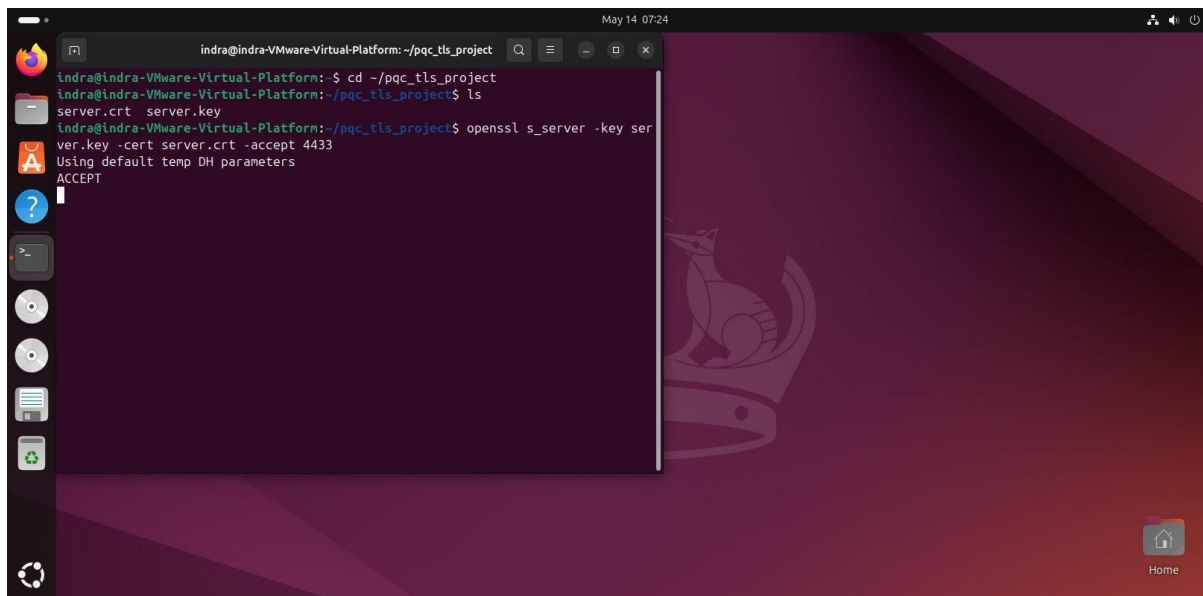


Figure: TLS server listening on port 4433 using the PQC-derived key pair (server.key / server.crt).

8.9 TLS Session Established with Session Ticket

After a client connected, the server printed the full set of session parameters it had negotiated. The screenshot below shows the TLS session ticket hex dump, the session ticket lifetime hint of 7200 seconds, the start time, the timeout, the message “Verify return code: 18 (self-signed certificate)” (expected here because the certificate was self-signed), and “Extended master secret: no, Max Early Data: 0”. The presence of the session ticket and the timing block proves that the TLS handshake has completed end-to-end and that the connection is now in the encrypted application-data phase, with read R BLOCK indicating the server is waiting for the next record from the client.

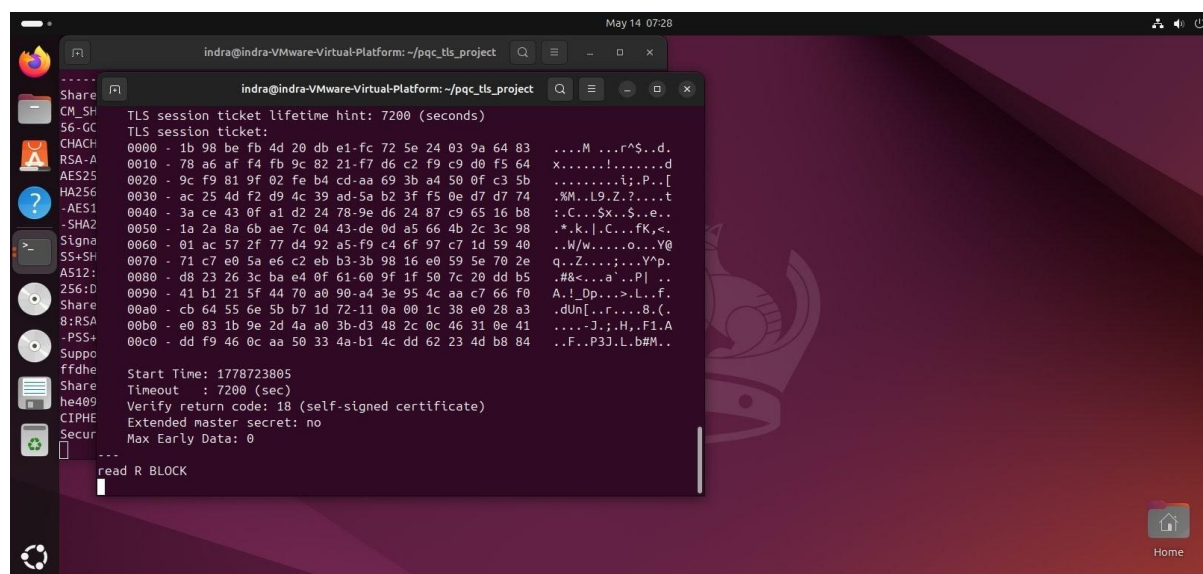


Figure: Server-side view of the established TLS session, including the session ticket and verification result

8.10 Client-Server Encrypted Communication

To confirm that the secured channel was functional, a short message was exchanged between client and server. The left terminal is the `s_client` side and the right terminal is the `s_server` side. The negotiated shared ciphers include `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256` and `TLS_AES_128_GCM_SHA256`; the supported and shared groups list `x25519`, `secp256r1`, `x448`, `secp521r1`, `secp384r1`, `ffdhe2048`, `ffdhe3072`, `ffdhe4096`, `ffdhe6144` and `ffdhe8192`; and the line `CIPHER is TLS_AES_256_GCM_SHA384` confirms the final selection. The plaintext `test123` typed in one terminal appears in the other, confirming that the application data is being encrypted, transmitted and decrypted correctly through the PQC-enabled TLS channel.

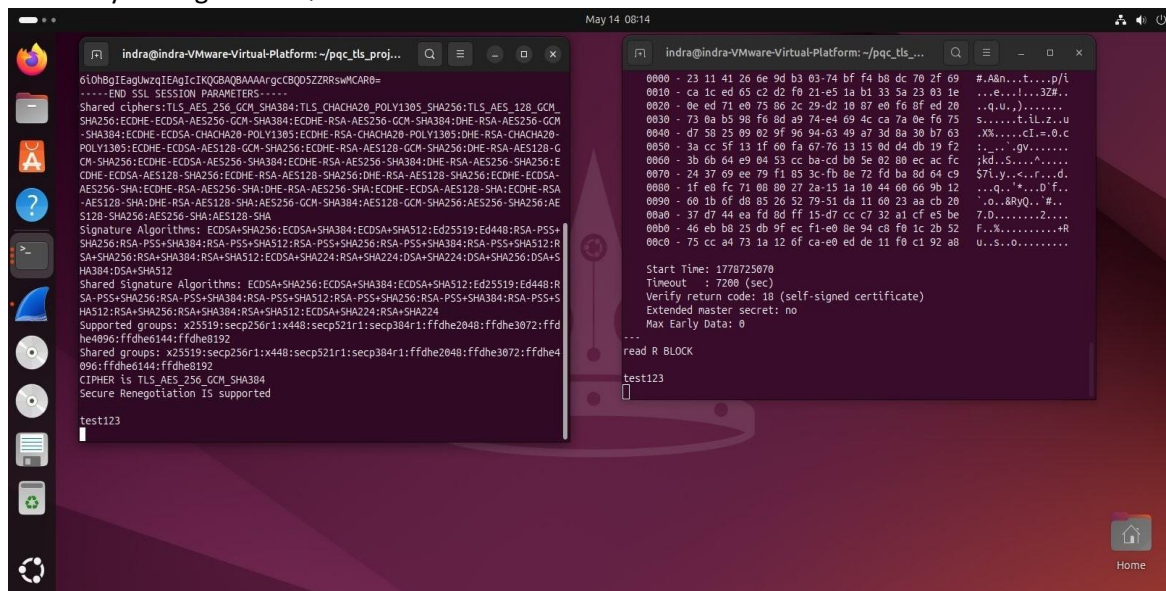


Figure: Bidirectional encrypted message exchange between OpenSSL `s_client` (left) and `s_server` (right) over the PQC-enabled TLS session.

8.11 Initial TLS Packet Capture

After the PQC-enabled OpenSSL stack was running, a simple TLS exchange was performed between a local server and client. This first Wireshark capture shows the initial encrypted traffic on the loopback interface `127.0.0.1`, filtered using the `tls` display filter. The two visible frames are Application Data records, which means the TLS handshake had already completed and the channel had moved into encrypted payload exchange.

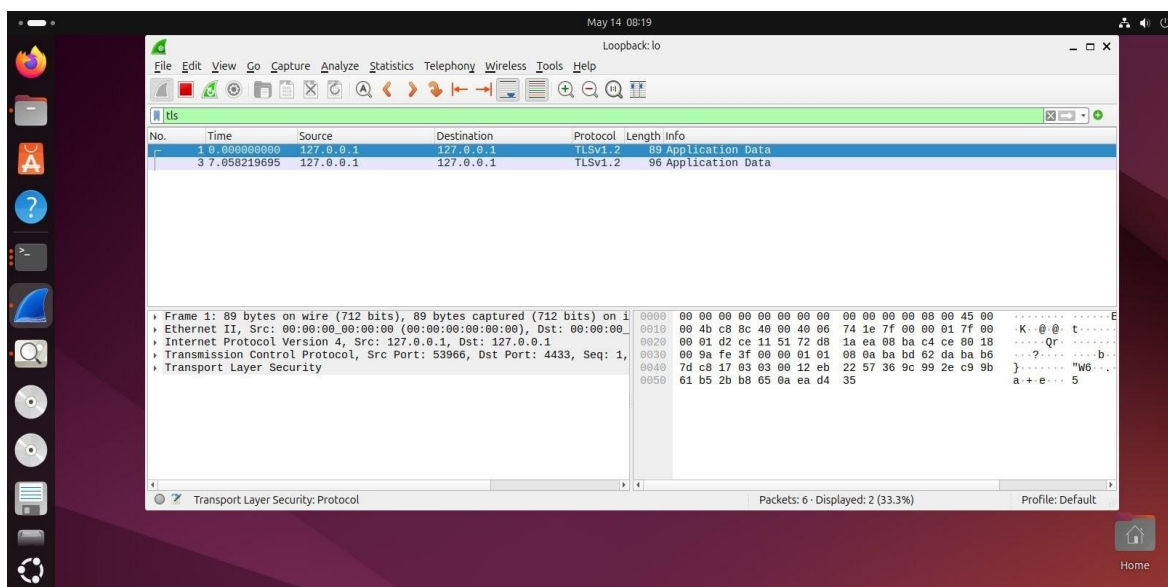


Figure: Initial Wireshark capture showing encrypted TLS Application Data on loopback

8.12 Cipher Suite Negotiation Verification

To make the parameter negotiation itself easy to inspect, the `s_client` output was captured and placed side-by-side with the `s_server`'s session ticket dump. The shared ciphers list, signature algorithms list, supported groups, shared groups and the final selected cipher (CIPHER is TLS_AES_256_GCM_SHA384) are all visible on the left, while the right panel shows the resulting session ticket bytes and the timing block (Start Time, Timeout 7200 sec, Verify return code 18). Together these two views demonstrate that the cipher suite, the signature algorithm and the key-exchange group were all agreed upon before any application data was exchanged — an essential property of any correctly implemented TLS endpoint.

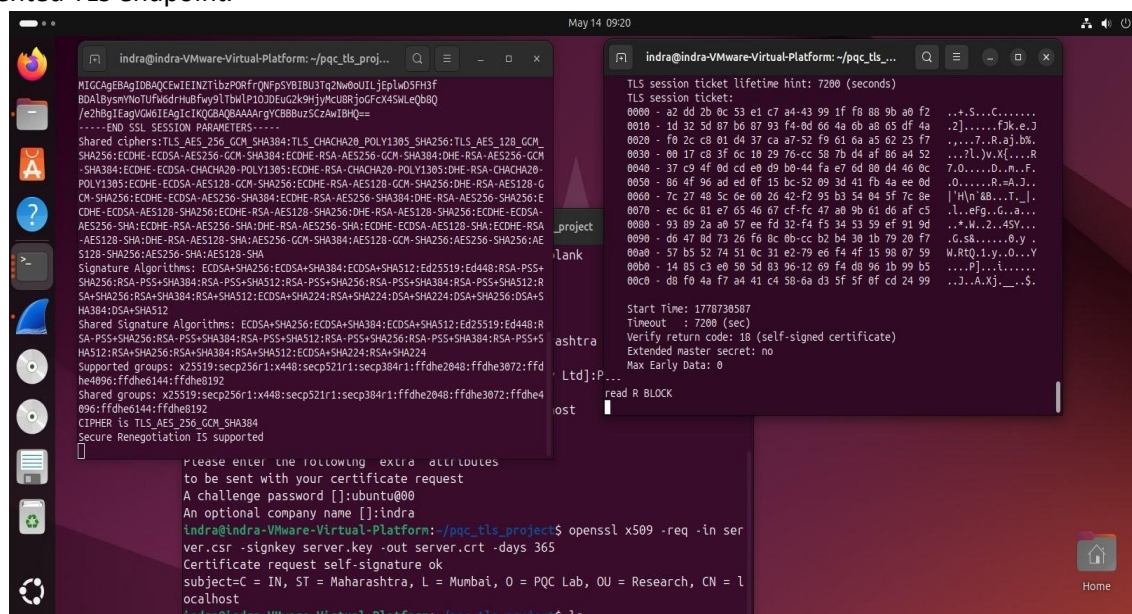


Figure: Cipher suite, signature algorithm and group negotiation as reported by `s_client` and `s_server`

8.13 Full TLS 1.3 Handshake Analysis

The next capture session showed a full TLS handshake. In this screenshot the important handshake messages are clearly visible: the Client Hello at frame 22, the combined Server Hello, Change Cipher Spec, Application Data in frame 24, and the subsequent Application Data records exchanged afterwards. The capture uses TLS 1.3, which is the modern version that supports hybrid PQC key exchange. The handshake size (notice frame 24 at 1491 bytes) is noticeably larger than in classical TLS, which reflects the bigger key shares carried by post-quantum algorithms.

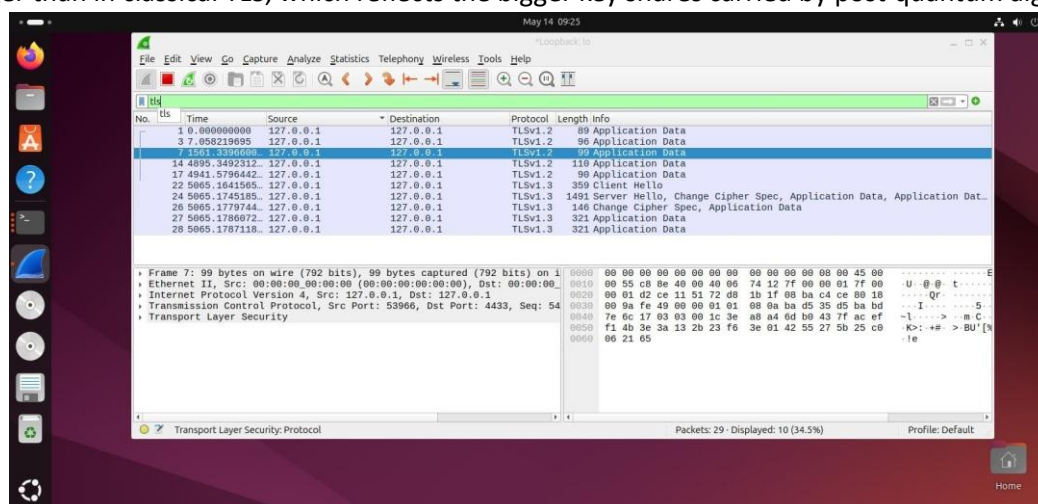


Figure: Full TLS 1.3 handshake with Client Hello, Server Hello, Change Cipher Spec and Application Data.

9 Performance Evaluation and Comparison

A comparison was performed between Classical TLS and PQC-based TLS communication to understand the impact of Post-Quantum Cryptography on secure communication systems. The evaluation focused on handshake size, computational overhead, and the strength of security against both classical and quantum adversaries.

Classical TLS communication generally provides faster handshakes because traditional algorithms (RSA, ECDHE) use smaller keys and lower computational resources. However, these algorithms may become vulnerable against future quantum computers.

PQC-based TLS communication introduces larger handshake sizes and slightly higher computational overhead. During implementation, it was observed in Wireshark that PQC handshake frames were significantly larger than classical ones — in some captures the server’s combined Hello message reached around 1.4 KB, compared to a few hundred bytes for traditional TLS.

Even though PQC introduces additional overhead, it provides much stronger future-ready protection against quantum attacks, which justifies the trade-off for systems that need long-term confidentiality

Comparison Summary

Feature	Classical TLS	PQC-based TLS
Security against classical computers	Strong	Strong
Security against quantum computers	Weak	Strong
Handshake size	Smaller	Larger
Computational overhead	Lower	Higher
Future readiness	Limited	High
Compatibility	Widely supported	Developing

10 Conclusion

This project successfully demonstrated secure TLS communication integrated with Post-Quantum Cryptography using OpenSSL, liboqs, and the OQS provider.

The implementation included repository setup, compilation of PQC libraries, TLS certificate generation, server–client communication, and Wireshark packet analysis. The TLS handshake and encrypted communication were successfully verified during packet capture analysis, where both Client Hello / Server Hello messages and encrypted Application Data could be observed clearly.

The comparison between Classical TLS and PQC-based TLS showed that PQC introduces slightly higher overhead but provides significantly better protection against future quantum threats.

Overall, the project demonstrated that Post-Quantum Cryptography is an important step toward building future-ready cybersecurity systems capable of resisting both classical and quantum attacks.

11 Future Scope

In the future, this implementation can be extended using cloud infrastructure, real-world web servers (such as Nginx and Apache with PQC modules), and advanced hybrid cryptographic configurations that combine multiple PQC algorithms.

Further optimization can also be performed to reduce handshake size and improve performance efficiency, for example by tuning record fragmentation and choosing lighter PQC variants where possible. As quantum computing technology continues to develop, PQC-based TLS communication may become an important standard for securing internet communication worldwide.